

SimPy 接入 API 实现模型自动生成 — 使用说明

项目目录：F:\Simpy | 生成日期：2026-09-07

一、项目背景

项目目录下有两部分内容：

1. 顶层散落的入门文件 (SimPy\SimPy\)

- `SimPy_Test.py` — 最基础的 SimPy 汽车延时示例。
- `api_test.py` — 一个最简单的 FastAPI 示例（只回显参数，未接仿真）。
- `sim_api.py` — 早期草稿，其中 import 的 `config` / `simulation` 模块并不在旁边，真正的实现在 `simpy-kpi-lab` 包中。

2. 核心项目 simpy-kpi-lab (版本 0.2.0)

位于 `SimPy\SimPy\simpy-kpi-lab\simpy-kpi-lab\`，虚拟环境已装好（simpy 4.1.2、openai 2.54.0、fastapi、uvicorn，`simlab.exe` 可直接使用）。整体架构：

YAML/JSON 配置 → SimPy 排队仿真（replications + 参数网格）→ KPI 统计
→ ① OpenAI 结构化 KPI 分析（`ai.py`）
→ ② OpenAI 白名单动作提案（`agent.py`）→ 人工审批（`workflow.py`）→ 新配置版本 → 再仿真（闭环）

各模块职责：

文件	职责
<code>config.py</code>	YAML 严格校验（Pydantic, <code>extra="forbid"</code> ）：工位、分布、实验、OpenAI 设置
<code>simulation.py</code>	SimPy 模型本体（串行多工位排队系统）
<code>ai.py</code>	<code>OpenAIKPIAnalyst</code> ：Responses API <code>responses.parse(text_format=KPIAnalysis)</code> 输出结构化 KPI 分析
<code>agent.py</code>	<code>OpenAIProposalAgent</code> ：根据当前配置 + KPI 生成待审批的结构化动作提案
<code>workflow.py</code>	版本化人工审批状态机（pending / approved / rejected / applied / failed + 审计日志 + 幂等）
<code>service.py</code>	会话编排：提案 → 审批 → 用新配置快照再跑仿真

二、关键概念：“模型自动生成”指什么

AI 不写 SimPy 代码、不执行任何东西——这是该项目的核心安全设计。所谓“自动生成模型”是指：AI 根据仿真 KPI 结果，自动生成**下一轮模型的结构化参数提案**（如把 `stations.1.capacity` 从 2 改成 3、启用某资源、改策略），人批准后由**本地代码**应用到新的不可变配置快照上，再跑下一轮仿真，形成闭环。

三类动作是白名单限定的：`set_parameter`、`change_policy`、`enable_resource`，且 `requires_approval` 恒为 `true`（必须人工审批）。

三、怎么使用

前置条件只有两个：Python 3.11+（已满足）和 `OPENAI_API_KEY` 环境变量（项目目录下的《SimPy_KPI_Lab_OpenAI_API获取与接入详细教程.docx》是获取 key 的详细教程；key 只从环境变量读取，不写入任何文件）。

方式一：CLI 一步分析（最简单）

```
cd F:\SimPy\SimPy\SimPy\simpy-kpi-lab\simpy-kpi-lab

# 只设置一次
$env:OPENAI_API_KEY = "sk-..."

# 1. 校验配置（3 个场景 × 20 次 = 60 次仿真）
.\.venv\Scripts\simlab.exe validate examples\service_center.yaml

# 2. 跑仿真 + 自动调 OpenAI 分析 KPI
.\.venv\Scripts\simlab.exe run examples\service_center.yaml --workers 4 --analyze

# 3. 或对已有结果单独分析，可带业务问题
.\.venv\Scripts\simlab.exe analyze outputs\service_center\results.json `
--question "比较各容量方案的服务水平、P95 周期时间和利用率，并指出权衡。"
```

第 2、3 步会生成 `ai_analysis.json` 和 `ai_analysis.md` 两个分析文件。

方式二：安全闭环控制 API（真正的“AI 自动生成下一轮模型”）

```
# 设置 Token 并启动服务（默认只监听本机 8000 端口）
$env:SIMLAB_API_TOKEN = "随便一个长随机串"
$env:SIMLAB_API_OUTPUT_ROOT = "outputs/api_sessions"
.\.venv\Scripts\simlab.exe serve
```

然后打开 `http://127.0.0.1:8000/docs` (OpenAPI 交互页面) , 按顺序调用:

1. `POST /v1/sessions` — 用配置 JSON (可用 `examples\api_service_center.json`) 创建会话, 记下 `workflow_version` (每步都要带当前版本号)。
2. `POST /v1/sessions/{id}/runs` — 跑一轮实验。
3. `GET /v1/sessions/{id}/allowed-actions` — 查看当前动作白名单。
4. `POST /v1/sessions/{id}/proposals:generate` — **这一步就是接入 OpenAI**: 把最近一次运行的 KPI 汇总 + 当前配置 + 白名单发给模型, 模型返回结构化提案计划 (可多步动作)。代理会先本地 dry-run 校验每个动作后才入库。
5. `POST /v1/sessions/{id}/proposals/{pid}:approve` 或 `:reject` — 人工逐条审批; 批准后本地代码应用到新配置版本。
6. 回到第 2 步用新配置再跑, 形成闭环; `GET /v1/sessions/{id}/audit` 可查全量审计。

PowerShell 端到端演示 (人工提案版): `.\examples\human_in_the_loop.ps1`。每一步写操作都要求唯一的 `operation_id` 和当前 `expected_version`, 旧版本会得到 409 冲突而不是覆盖。

方式三: MCP 接口 (给模型用的受限入口)

```
$env:SIMLAB_API_URL = "http://127.0.0.1:8000"
$env:SIMLAB_API_TOKEN = "与 REST 相同的 Token"
.\.venv\Scripts\simlab-mcp.exe
```

只暴露 `validate_project`、`get_session`、`get_allowed_actions`、`submit_proposal`、`list_events`, **不暴露**建会话 / 运行 / 批准 / 拒绝——模型无法绕过人工审批。

四、几点提醒

- **默认模型是 YAML 里的 `gpt-5.6`** (config.py 中 OpenAIConfig.model 的默认值), 可用环境变量 `SIMLAB_OPENAI_MODEL` 或 CLI 的 `--model` 参数覆盖成账号可用的模型; 超时 60s、重试 2 次、`store: false` 均可在 YAML 的 `openai:` 段配置。
- 不带 `--analyze` 的仿真完全不需要 API key; AI 层是可选的, KPI 数值全部由本地代码计算, 模型只能解读数据, 不会改写实验结果。
- 会话 / 审计存在单进程内存里, 服务重启即丢失 (生产部署需换 SQLite / PostgreSQL)。
- 想扩展模型: 改 `simulation.py` 里的 `customer()` 加路由 / 返工 / 优先级; 在 `config.py` 加业务参数; 在 KPI 收集器加指标——但不要让 AI 参与 KPI 数值计算。

安全边界小结: 模型收到的只有“当前配置 + KPI 汇总 + 业务目标 + 允许动作目录”, 返回后还要经过本地白名单逐字校验、`requires_approval=true` 检查、完整 Pydantic 配置校验和配置哈希新鲜度检查; 模型不能设置 API key、模型名、输出路径、随机 seed 或任意路径, 也没有

shell / 文件 / eval / exec 工具。批准动作只影响下一次运行，正在运行的 SimPy replication 不会被热修改，保证 seed、配置与结果可审计复现。